

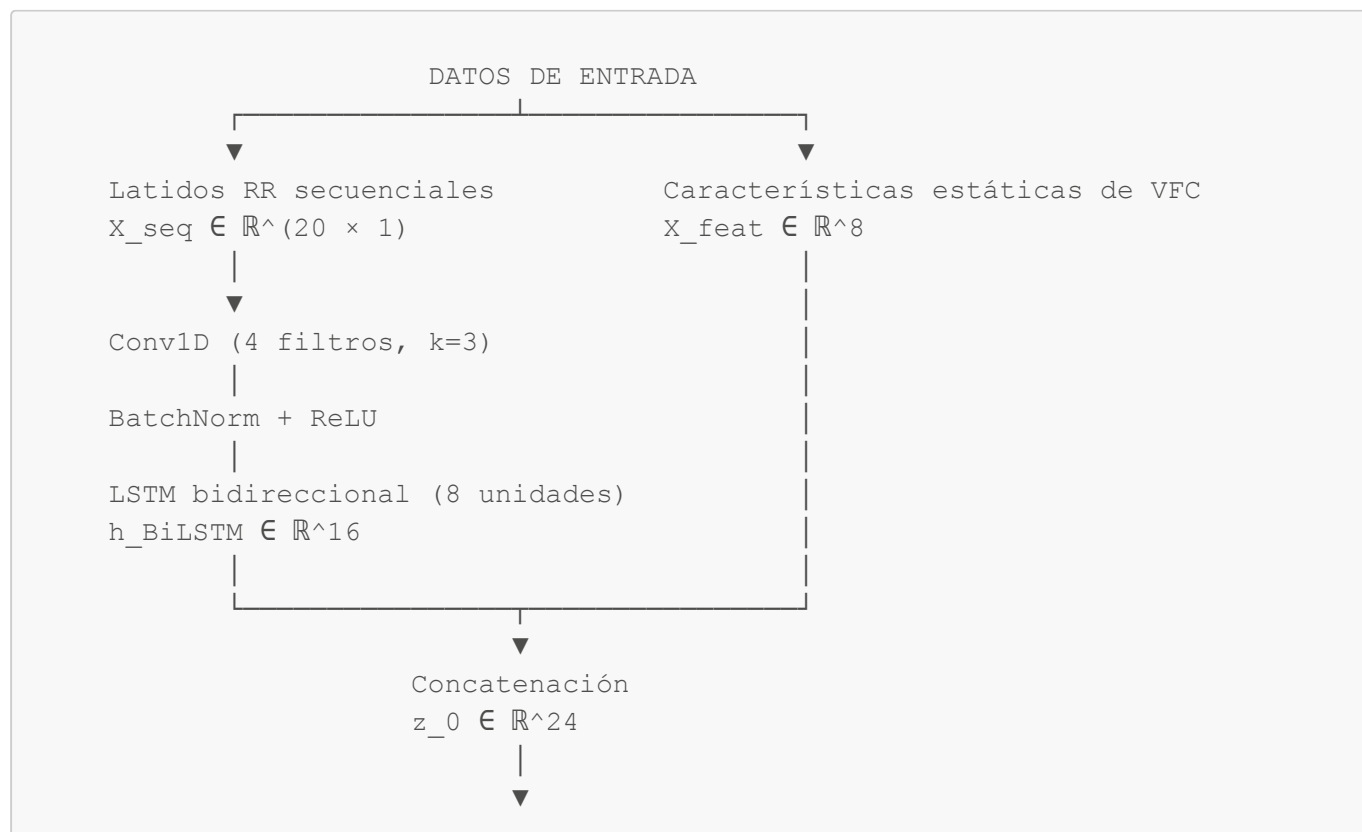
Informe técnico: Arquitectura de red neuronal híbrida profunda para el pronóstico de la variabilidad de la frecuencia cardíaca (ΔRR)

1. Visión general de la arquitectura y diseño del sistema

El objetivo del modelo es predecir la diferencia latido a latido inmediata en milisegundos ($\Delta RR_{20 \rightarrow 21} = RR_{21} - RR_{20}$) entre el último intervalo observado en una ventana de 20 latidos y el intervalo subsiguiente.

Para capturar tanto la dinámica secuencial temporal local (transiciones de latidos a corto plazo) como el contexto autonómico / del espacio de fases global (índices estadísticos y no lineales de VFC), la red implementa una arquitectura híbrida de dos ramas:

- **Rama secuencial (Conv1D $\rightarrow$ BiLSTM):** Recibe la secuencia estandarizada de 20 latidos $\mathbf{X}_{seq} \in \mathbb{R}^{20 \times 1}$. Utiliza convoluciones 1D para extraer características morfológicas locales de tres latidos (*tri-beat*), seguidas de una LSTM bidireccional para capturar dependencias temporales bidireccionales.
- **Rama de características contextuales (características estáticas de VFC):** Recibe 8 métricas de resumen obtenidas por ingeniería de características del dominio $\mathbf{X}_{feat} \in \mathbb{R}^8$ (período cardíaco medio local, dispersión en el dominio del tiempo, geometría de Poincaré, correlación no lineal e índices de asimetría).
- **Cabezal de fusión y regresión (MLP denso):** Concatena las representaciones recurrentes aprendidas con las características de VFC diseñadas manualmente, procesándolas a través de un perceptrón multicapa con normalización por lotes (*Batch Normalization*), activaciones ReLU y *Dropout* para predecir el delta escalar.



```

Dense(24) + BN + ReLU + Dropout(0.2)
    |
    ▼
Dense(16) + BN + ReLU + Dropout(0.2)
    |
    ▼
Dense(1, Linear)
    |
    ▼
ΔRR predicho (ms)

```

2. Formulación matemática de las capas de la red

2.1. Rama temporal secuencial

Paso 1: Extracción de características locales mediante convolución 1D

Sea la secuencia de entrada $\mathbf{X}_{\text{seq}} = [x_1, x_2, \dots, x_{20}]^T \in \mathbb{R}^{20 \times 1}$, donde cada x_t representa un intervalo entre latidos estandarizado por edad.

Una capa convolucional 1D con $F = 4$ filtros y una longitud de núcleo (*kernel*) $K = 3$ (relleno válido / *valid padding*, paso / *stride* $s = 1$) recorre la secuencia. Para el filtro $j \in \{1, \dots, 4\}$ y el índice temporal local $t \in \{1, \dots, 18\}$:

$$c_j(t) = \sum_{k=0}^2 w_{j,k} \cdot x_{t+k} + b_j$$

donde $\mathbf{w}_j \in \mathbb{R}^3$ y $b_j \in \mathbb{R}$ son los pesos del núcleo y el sesgo (*bias*) entrenables. Esta operación extrae patrones locales de aceleración/desaceleración de tres latidos.

Paso 2: Normalización por lotes y activación

Los mapas de características convolucionales se normalizan a lo largo del minilote $\mathcal{B}$:

$$\hat{c}_j(t) = \frac{c_j(t) - \mu_{\mathcal{B},j}}{\sqrt{\sigma_{\mathcal{B},j}^2 + \epsilon_{\text{BN}}}}, \quad y_j(t) = \gamma_j \hat{c}_j(t) + \beta_j$$

seguido de una activación punto a punto por unidad lineal rectificada (ReLU):

$$v_j(t) = \text{ReLU}(y_j(t)) = \max(0, y_j(t))$$

obteniendo una matriz de secuencia intermedia $\mathbf{V} = [\mathbf{v}(1), \dots, \mathbf{v}(18)]^T \in \mathbb{R}^{18 \times 4}$.

Paso 3: Modelado recurrente bidireccional (BiLSTM)

Para capturar dependencias que dependen tanto de las transiciones locales precedentes como de las sucesivas dentro de la ventana, la secuencia de características $\mathbf{V}$ se transmite a una LSTM bidireccional con $U = 8$ unidades ocultas en cada dirección:

- **LSTM hacia adelante ($\overrightarrow{\text{LSTM}}$):** Procesa $\mathbf{v}(1) \rightarrow \mathbf{v}(18)$, produciendo el estado oculto $\overrightarrow{\mathbf{h}}_{18} \in \mathbb{R}^8$.
- **LSTM hacia atrás ($\overleftarrow{\text{LSTM}}$):** Procesa $\mathbf{v}(18) \rightarrow \mathbf{v}(1)$, produciendo el estado oculto $\overleftarrow{\mathbf{h}}_1 \in \mathbb{R}^8$.

La representación latente secuencial final es la concatenación de los estados ocultos terminales:

$$\mathbf{h}_{\text{BiLSTM}} = [\overrightarrow{\mathbf{h}}_{18} \parallel \overleftarrow{\mathbf{h}}_1] \in \mathbb{R}^{16}$$

2.2. Rama de características específicas del dominio y fusión

El vector de entrada estático $\mathbf{X}_{\text{feat}} \in \mathbb{R}^8$ contiene las características estadísticas y no lineales del dominio precalculadas:

$$\mathbf{X}_{\text{feat}} = [\text{mean}, \text{sdsd}, \text{sd2}, \text{ccm}, \text{guzik}, \text{nn50}, \text{porta}, \text{std}]^\top$$

La representación secuencial aprendida $\mathbf{h}_{\text{BiLSTM}}$ y las características de dominio diseñadas $\mathbf{X}_{\text{feat}}$ se fusionan mediante concatenación a nivel de características:

$$\mathbf{z}_0 = [\mathbf{h}_{\text{BiLSTM}} \parallel \mathbf{X}_{\text{feat}}] \in \mathbb{R}^{24}$$

2.3. Cabezal de regresión de perceptrón multicapa (MLP)

El vector fusionado $\mathbf{z}_0$ pasa por dos bloques de regularización totalmente conectados:

Bloque denso 1 (Dimensionalidad: $24 \rightarrow 24$):

$$\mathbf{a}_1 = \mathbf{W}_1 \mathbf{z}_0 + \mathbf{b}_1, \quad \mathbf{W}_1 \in \mathbb{R}^{24 \times 24}$$

$$\hat{\mathbf{a}}_1 = \text{BatchNorm}(\mathbf{a}_1)$$

$$\mathbf{z}_1 = \text{Dropout}_{0.2}(\text{ReLU}(\hat{\mathbf{a}}_1))$$

Bloque denso 2 (Dimensionalidad: $24 \rightarrow 16$):

$$\mathbf{a}_2 = \mathbf{W}_2 \mathbf{z}_1 + \mathbf{b}_2, \quad \mathbf{W}_2 \in \mathbb{R}^{16 \times 24}$$

$$\hat{\mathbf{a}}_2 = \text{BatchNorm}(\mathbf{a}_2)$$

$$\mathbf{z}_2 = \text{Dropout}_{0.2}(\text{ReLU}(\hat{\mathbf{a}}_2))$$

Proyección lineal de salida: La predicción escalar $\hat{y} \in \mathbb{R}$ se calcula mediante una proyección lineal final:

$$\hat{y} = \mathbf{w}_{\text{out}}^\top \mathbf{z}_2 + b_{\text{out}}, \quad \mathbf{w}_{\text{out}} \in \mathbb{R}^{16}, b_{\text{out}} \in \mathbb{R}$$

3. Resumen de dimensionalidad de capas y flujo de tensores

Etapla / Capa	Forma del tensor de entrada	Forma del tensor de salida	Activación / Operación	Propósito principal
Entrada (Secuencial)	—	$(B, 20, 1)$	Ninguna	Recibe la secuencia RR de 20 latidos
Entrada (Estática)	—	$(B, 8)$	Ninguna	Recibe 8 métricas de VFC del dominio
Conv1D	$(B, 20, 1)$	$(B, 18, 4)$	Lineal ($k = 3, valid$)	Extrae micropatrones locales de 3 latidos
BatchNorm + ReLU	$(B, 18, 4)$	$(B, 18, 4)$	$\text{ReLU}(z)$	Normaliza las distribuciones por canal
LSTM bidireccional	$(B, 18, 4)$	$(B, 16)$	$\tanh / \sigma$	Codificación secuencial recurrente (8 fwd + 8 bwd)
Concatenación	$(B, 16) + (B, 8)$	$(B, 24)$	Fusión (<i>Merge</i>)	Unifica representaciones aprendidas y manuales
Bloque denso 1	$(B, 24)$	$(B, 24)$	BN + ReLU + Drop(0.2)	Captura interacciones no lineales de características
Bloque denso 2	$(B, 24)$	$(B, 16)$	BN + ReLU + Drop(0.2)	Compresión latente abstracta
Capa de salida	$(B, 16)$	$(B, 1)$	Lineal	Predice el delta escalar $\widehat{\Delta RR}$ (ms)

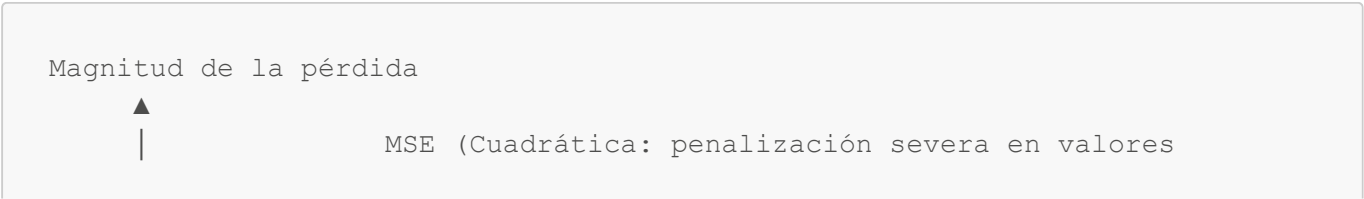
Nota: $B = 2048$ denota el tamaño del lote (batch size).

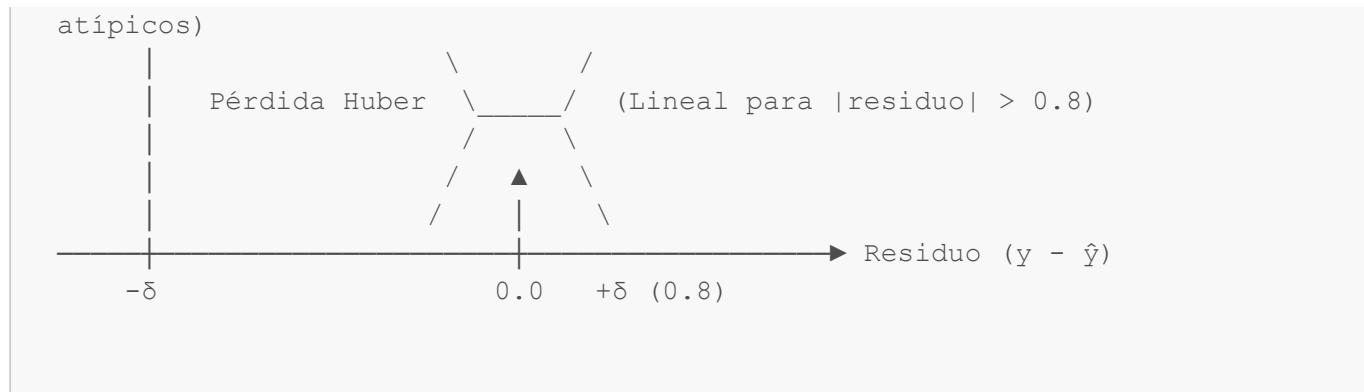
4. Formulación de la función de pérdida y robustez frente a valores atípicos (Pérdida de Huber)

Las series RR cardíacas contienen por naturaleza ruido no gaussiano, tales como latidos ectópicos, picos de arritmia sinusal respiratoria y artefactos menores de medición. El entrenamiento con el error cuadrático medio estándar (MSE) puede desestabilizar los gradientes debido a las penalizaciones cuadráticas sobre los residuos extremos.

Para equilibrar la sensibilidad a ajustes finos con la robustez frente a valores atípicos (*outliers*), el modelo se optimiza mediante la pérdida de Huber (*Huber Loss*) con un umbral $\delta = 0.8$:

$$L_{\delta}(y, \hat{y}) = \begin{cases} \frac{1}{2}(y - \hat{y})^2 & \text{si } |y - \hat{y}| \leq \delta \\ \delta \cdot |y - \hat{y}| - \frac{1}{2}\delta^2 & \text{si } |y - \hat{y}| > \delta \end{cases}$$





Dinámica de gradientes:

El gradiente con respecto al residuo $r = y - \hat{y}$ demuestra el mecanismo de estabilización:

$$\frac{\partial L_{\delta}}{\partial \hat{y}} = \begin{cases} -(y - \hat{y}) & \text{si } |y - \hat{y}| \leq 0.8 \quad (\text{lineal, el gradiente escala con el error}) \\ -0.8 \cdot \text{sgn}(y - \hat{y}) & \text{si } |y - \hat{y}| > 0.8 \quad (\text{constante, gradientes acotados}) \end{cases}$$

Esto garantiza que los errores pequeños se minimicen de forma suave, al tiempo que los latidos atípicos no provocan explosiones de gradiente.

5. Balanceo de clases y cohortes mediante ponderación de muestras por frecuencia inversa

Al entrenar con múltiples intervalos de registro $\mathcal{K} = \{1, \dots, K\}$, la disparidad en las duraciones de registro de los sujetos puede causar que las cohortes más grandes dominen la función de pérdida.

Para imponer una representación equilibrada en todos los intervalos experimentales sin descartar datos, se aplican pesos estáticos de muestra por frecuencia inversa durante el cálculo de la pérdida.

Para cada muestra i perteneciente al intervalo $k(i) \in \mathcal{K}$:

$$w_i = \frac{N_{\text{total}}}{K \cdot N_{k(i)}}$$

donde:

- N_{total} es el número total de muestras de entrenamiento en todos los intervalos.
- K es el número total de intervalos únicos.
- $N_{k(i)}$ es la cantidad de muestras en el intervalo $k(i)$.

La pérdida ponderada del lote minimizada por el optimizador es:

$$\mathcal{L}_{\mathcal{B}} = \frac{\sum_{i \in \mathcal{B}} w_i \cdot L_{\delta}(y_i, \hat{y}_i)}{\sum_{i \in \mathcal{B}} w_i}$$

Esto asegura que cada grupo de intervalos contribuya de manera equitativa a las actualizaciones de parámetros a lo largo de cada época completa.

6. Régimen de entrenamiento, optimización y regularización

6.1. Configuración de optimización

- **Optimizador:** Adam ($\alpha = 0.005, \beta_1 = 0.9, \beta_2 = 0.999, \epsilon = 10^{-7}$).
- **Tamaño del lote (Batch Size):** $B = 2048$, cargado directamente en la memoria de la GPU para un alto rendimiento y estimaciones de gradiente de minilote estables.
- **Épocas máximas:** 100 con mezcla aleatoria (*shuffling*) del conjunto de datos en cada época.

6.2. Dinámica de la tasa de aprendizaje y control de convergencia

- **Decaimiento de la tasa de aprendizaje en meseta (*ReduceLROnPlateau*):** Monitorea la pérdida de Huber en validación ($\mathcal{L}_{\text{val}}$). Si no se observa mejoría durante 4 épocas consecutivas, la tasa de aprendizaje se reduce por un factor $\gamma = 0.5$:

$$\alpha_{t+1} = \max(0.5 \cdot \alpha_t, 10^{-6})$$

- **Parada temprana (*Early Stopping*):** Monitorea la pérdida de validación con una paciencia de 10 épocas. Al finalizar, los pesos del modelo se restauran a aquellos correspondientes a la menor pérdida de validación registrada:

$$\theta^* = \arg \min_{\theta_e} \mathcal{L}_{\text{val}}(e)$$

- **Puntos de control del modelo (*Model Checkpointing*):** Guarda de manera persistente el modelo con mejor rendimiento (`cnn_lstm_hrv_best.keras`) en función de la mínima pérdida de validación.

7. Resumen de la justificación fisiológica y metodológica

- **Desacoplamiento de corto vs. largo alcance:** La capa Conv1D aísla las aceleraciones/desaceleraciones localizadas de 3 latidos, mientras que la BiLSTM integra la progresión secuencial a lo largo de todo el contexto de 20 latidos.
- **Preservación del espacio de fases:** Métricas no lineales como SD_2 , CCM e índices de asimetría (*Guzik, Porta*) resumen propiedades de la trayectoria en el espacio de fases que la recurrencia estándar podría tener dificultades para inferir únicamente a partir de los intervalos sin procesar. Introducir las directamente en la capa de fusión previene la pérdida de estas relaciones globales.
- **Mitigación del sobreajuste en señales biológicas:** La normalización por lotes en dos etapas combinada con un *Dropout* moderado ($p = 0.2$) evita la dependencia excesiva de patrones de latidos individuales, mejorando la generalización entre diferentes sujetos.